

RealMan Robotic Arm rm_driver Topic Detailed Description (ROS2) V1.1.6

RealMan Intelligent Technology (Beijing) Co., Ltd.

Revision History

No.	Date	Comment
V1.0	2024-2-18	Draft
V1.1	2024-7-8	Amend(Add teaching instructions3.6)
V1.1.1	2024-8-13	Amend(Add six-axis topic)
V1.1.2	2024-9-25	Amend(revise coordinate topic description)
V1.1.3	2024-10-31	Amend(Add Agile Hand UDP Adaptation,Follow Adaptation)
V1.1.4	2024-12-25	Amend(Modify UDP report content)
V1.1.5	2025-02-19	Amend(API2 Adaptation; add end ecosystem protocol interface; update UDP interface)
V1.1.6	2025-05-26	Revision (adapted to fourth generation controllers, added version query interface, added Cartesian space linear offset motion interface, added Modbus interface, added trajectory list interface)

Content

- 1.Introduction
- 2.Error Description
- 2.1Controller error type
- 2.2Joint error type
- 2.3API error type
- 3.ROS Function Package Robotic Arm Instructions
- 3.1Joint Configuration
- 3.1.1Clear the joint's error code
- 3.2Get version
- 3.2.1Get basic information of robotic arm
- 3.2.2Get the software information of the robotic arm
- 3.2.3Get the joint software version
- 3.2.4Get the software version of the end interface board
- 3.3Functions related to the work coordinate system settings
- 3.3.1Change the current work coordinate system
- 3.3.2Get the current work coordinate system
- 3.3.3Get all work coordinate system names
- 3.4Functions related to the tool coordinate system settings
- 3.4.1Change the current tool coordinate system
- 3.4.2Get the current tool coordinate system
- 3.4.3Get all tool coordinate system names
- 3.5Functions related to the arm state query
- 3.5.1Get the current state of the robot arm - return each joint angle + Euler angle
- 3.5.2Get the current state of the robotic arm - return each joint radians + quaternion
- 3.6Functions related to motion planning of the robotic arm
- 3.6.1Joint Space Motion
- 3.6.2Linear motion in Cartesian space
- 3.6.3Cartesian space linear offset motion
- 3.6.4Circular motion in Cartesian space
- 3.6.5Joint angle CANFD transmission
- 3.6.6Customize high following mode joint angle CANFD transmission
- 3.6.7Pose CANFD transmission
- 3.6.8Customize high following mode pose CANFD transmission
- 3.6.9Joint space planning to target pose
- 3.6.10Trajectory emergency stop
- 3.6.11Emergency stop
- 3.6.12Emergency pause
- 3.6.13Resume after trajectory pause
- 3.7Teaching instructions
- 3.7.1Joint teaching
- 3.7.2Position teaching
- 3.7.3Attitude teaching
- 3.7.4Stop teaching
- 3.8Trajectory File
- 3.8.1Query Trajectory List
- 3.8.2Start Running Specified Trajectory
- 3.8.3Delete Specified Trajectory File
- 3.8.4Save Trajectory File
- 3.8.5Query Flowchart Program Run State
- 3.9Modbus Configuration
- 3.9.1Set Controller RS485 Mode-Four_Generations

- 3.9.2Get Controller RS485 Mode-Four_Generations
- 3.9.3Set Tool End RS485 Mode-Four_Generations
- 3.9.4Get Tool End RS485 Mode-Four_Generations
- 3.9.5Set Controller And Tools ModbusRTU Mode-Three_Generations
- 3.9.6Close Controller And Tools ModbusRTU Mode-Three_Generations
- 3.10ModbusTCP Master
- 3.10.1Add Modbus TCP Master-Four_Generations
- 3.10.2Update Modbus TCP Master-Four_Generations
- 3.10.3Delete Modbus TCP Master-Four_Generations
- 3.10.4Get Specified Modbus TCP Master-Four_Generations
- 3.10.5Get Modbus TCP Master List-Four_Generations
- 3.10.6Set Modbus TCP Master-Three_Generations
- 3.10.7Close Modbus TCP Master-Three_Generations
- 3.11Tool end controller end RTU Modbus protocol reads and writes data
- 3.11.1Modbus RTU Protocol Read Coils-Four_Generations
- 3.11.2Modbus RTU Protocol Write Coils-Four_Generations
- 3.11.3Modbus RTU Protocol Read Discrete Inputs-Four_Generations
- 3.11.4Modbus RTU Protocol Read Holding Registers-Four_Generations
- 3.11.5Modbus RTU Protocol Write Holding Registers-Four_Generations
- 3.11.6Modbus RTU Protocol Read Input Registers-Four_Generations
- 3.11.7Modbus RTU Protocol Read Coils-Three_Generations
- 3.11.8Modbus RTU Protocol Write Coils-Three_Generations
- 3.11.9Modbus RTU Protocol Read Discrete Inputs-Three_Generations
- 3.11.10Modbus RTU Protocol Read Holding Registers-Three_Generations
- 3.11.11Modbus RTU Protocol Write Holding Registers-Three_Generations
- 3.11.12Modbus RTU Protocol Read Input Registers-Three_Generations
- 3.12Controller Modbus TCP protocol reads and writes data
- 3.12.1Modbus TCP Protocol Read Coils-Four_Generations
- 3.12.2Modbus TCP Protocol Write Coils-Four_Generations
- 3.12.3Modbus TCP Protocol Read Discrete Inputs-Four_Generations
- 3.12.4Modbus TCP Protocol Read Holding Registers-Four_Generations
- 3.12.5Modbus TCP Protocol Write Holding Registers-Four_Generations
- 3.12.6Modbus TCP Protocol Read Input Registers-Four_Generations
- 3.12.7Modbus TCP Protocol Read Coils-Three_Generations
- 3.12.8Modbus TCP Protocol Write Coils-Three_Generations
- 3.12.9Modbus TCP Protocol Read Discrete Inputs-Three_Generations
- 3.12.10Modbus TCP Protocol Read Holding Registers-Three_Generations
- 3.12.11Modbus TCP Protocol Write Holding Registers-Three_Generations
- 3.12.12Modbus TCP Protocol Read Input Registers-Three_Generations
- 3.13Functions related to the IO configuration of the end tool
- 3.13.1Setting the tool voltage output
- 3.14Functions related to the control of the end gripper - optional
- 3.14.1Setting the Gripper Pick
- 3.14.2Setting the gripper pick-on
- 3.14.3Setting the gripper to the given position
- 3.15Functions related to the drag teach and trajectory reproduction
- 3.15.1Set the force-position mixing control
- 3.15.2Stop the force-position mixing control
- 3.16Functions related to the use of six-axis force sensors at the end - optional
- 3.16.1Query the six-axis force data
- 3.16.2Clearing the six-axis force Data
- 3.17Functions related to the control of the five-finger dexterous hand - optional

- 3.17.1Setting the serial number of the dexterous hand posture
- 3.17.2Set the dexterous hand action sequence number
- 3.17.3Setting the angles of various degrees of freedom for the dexterous hand
- 3.17.4Setting the dexterous hand speed
- 3.17.5Setting the force threshold of the dexterous hand
- 3.17.6Setting the angle following of the dexterous hand
- 3.17.7Setting the posture following of the dexterous hand
- 3.18Lifting mechanism
- 3.18.1Speed open-loop control of the lifting mechanism
- 3.18.2Position closed-loop control of the lifting mechanism
- 3.18.3Get the lifting mechanism state
- 3.19General expansion joint
- 3.19.1Get the state of the expansion joint
- 3.19.2Set the open-loop speed control of the expansion joint
- 3.19.3Set the closed-loop position control of the expansion joint
- 3.19End-Effector Ecosystem Command Set
- 3.19.1Setting End-Effector Ecosystem Protocol Mode
- 3.19.2Querying End-Effector Ecosystem Protocol Mode
- 3.19.3Setting Tactile Sensor Mode
- 3.19.4Querying Tactile Sensor Mode
- 3.20Functions related to the transmissive force-position compensation Mode
- 3.20.1Starting the transmissive force-position mixing control compensation mode
- 3.20.2Stopping the transmissive force-position mixing control compensation mode
- 3.20.3Transmissive force-position mixing control compensation - joint
- 3.20.4Transmissive force-position mixing control compensation - pose
- 3.20.5Transmissive_force-position_mixing_control_compensation
- 3.21Robotic arm state active reporting
- 3.21.1Setting UDP robotic arm state active reporting configuration
- 3.21.2Getting UDP robotic arm state active reporting configuration
- 3.21.3UDP robotic arm state active reporting

Introduction

RealMan provides ROS2 function packages based on API to help users control the robotic arm using ROS2. If you want to learn more about controlling the robotic arm, you can refer to the API documentation and instructions. In practical use, the user can establish communication with the robotic arm through the Ethernet port and control the robotic arm.

Error_Description

Controller_error_type

Serial No.	Error code (hexadecimal)	Error content
1	0x0000	System is normal
2	0x1001	Joint communication is abnormal
3	0x1002	The target angle exceeds the limit
4	0x1003	This position is inaccessible and is a singular point
5	0x1004	Real-time kernel communication error
6	0x1005	Joint communication bus error
7	0x1006	Planning layer kernel error
8	0x1007	Joint Overspeed
9	0x1008	The end interface board

		cannot be connected
10	0x1009	Overspeed limit
11	0x100A	Overacceleration limit
12	0x100B	Joint brake is not opened
13	0x100C	Overspeed during drag teach
14	0x100D	Robotic arm collision
15	0x100E	No work coordinate system is available
16	0x100F	No tool coordinate system is available
17	0x1010	Joint failure enabling error

Joint_error_type

Serial No.	Error code (hexadecimal)	Error content
1	0x0000	Joint is normal
2	0x0001	FOC error
3	0x0002	Overvoltage
4	0x0004	Undervoltage
5	0x0008	Overtemperature
6	0x0010	Start failed
7	0x0020	Encoder error
8	0x0040	Overcurrent
9	0x0080	Software error
10	0x0100	Temperature sensor error
11	0x0200	Position limit-out error
12	0x0400	Illegal joint ID
13	0x0800	Position tracking error
14	0x1000	Current detection error
15	0x2000	Brake opening failed
16	0x4000	Position command step warning
17	0x8000	Multi-coil joint's coil lost the number
18	0xF000	Communication frame loss

API_error_type

Error Code (int)	Description	Resolution
0	System operating normally.	-
1	Message request returned FALSE.	- Verify JSON command: ① Enable API DEBUG logs to capture raw JSON data. ② Check JSON syntax: Ensure correct formatting of brackets, quotes, commas, etc. (use a JSON validation tool). ③ Refer to API documentation to validate parameter names, data types, and value ranges. ④ After fixing issues, resend the command and check the status code and business data returned by the controller. - Check robot

		arm status: ① Review real-time error messages in the robot arm controller or logs (e.g., hardware failures, limits exceeded), and reset, calibrate, or troubleshoot hardware issues as indicated. ② After resolving issues, resend the command and check the status code and business data returned by the controller.
-1	Data send failure, communication issue during transmission.	Check network connectivity: Use tools like ping/telnet to test the communication link with the controller.
-2	Data receive failure, communication issue during transmission or controller timeout without response.	- Check network connectivity: Use tools like ping/telnet to test the communication link with the controller. - Verify version compatibility: ① Confirm that the controller firmware version supports the current API functionality. Refer to the version change notes (https://develop.realman-robotics.com/robot4th/releaseNotes/releaseNotesfour/) for specific version compatibility. ② If the version is too low, upgrade the controller or use a compatible API version. - When calling ModbusTCP interface: Applicable only when reading/writing to the controller's ModbusTCP device. After creating the robot arm control handle, you must call the <code>rm_set_modbustcp_mode()</code> interface, otherwise, no return value will be received.
-3	Return value parsing failure, received data format is incorrect or incomplete.	Verify version compatibility: ① Confirm that the controller firmware version supports the current API functionality. Refer to the version change notes (https://develop.realman-robotics.com/robot4th/releaseNotes/releaseNotesfour/) for specific version compatibility. ② If the version is too low, upgrade the controller or use a compatible API version.

-4	Current device validation failed, i.e., the current device is not a joint/elevator/gripper/dexterous hand.	- Check for concurrent control of multiple devices: Ensure no other devices are sending motion commands to the robot arm, including movements of the robot arm, gripper, dexterous hand, and elevator. - Real-time monitoring of command events: Register the callback function <code>rm_get_arm_event_callback</code>: ① Capture device arrival events (e.g., motion completion, timeout); ② Use the device parameter in the callback to determine the specific type of device that triggered the event.
-5	Single-threaded blocking mode timed out without receiving a response, ensure the timeout setting is reasonable.	- Check timeout duration settings: In single-threaded blocking mode, you can configure the timeout for waiting for device motion completion. Ensure the timeout setting is greater than the device motion time. - Check network connectivity: Use tools like ping/telnet to test the communication link with the controller.

ROS_Function_Package_Robotic_Arm_Instructions

This section describes how to query and control the robotic arm through the topic of ROS.

Joint_Configuration

Clear_the_joint's_error_code

Function description	Clear_the_joint's_error_code
Parameter description	<code>Jointerrclear.msg</code> uint8 joint_num: The corresponding joint number, from the base to the end-effector of the robotic arm. For a 6-DoF arm, the joint numbers are sequentially 1 to 6. For a 7-DoF arm, the joint numbers are sequentially 1 to 7.
Command example	<pre>ros2 topic pub /rm_driver/set_joint_err_clear_cmd rm_ros_interfaces/msg/Jointerrclear "joint_num: 1 "</pre>
Return value	true-set successfully, false-set failed
Return example	<pre>ros2 topic echo /rm_driver/set_joint_err_clear_result</pre>

Get_version

Get_basic_information_of_robotic_arm

Function description	Get_basic_information_of_robotic_arm
Parameter description	std_msgs::msg::Empty.
Command example	ros2 topic pub --once /rm_driver/get_robot_info_cmd std_msgs/msg/Empty "{}"
Return value	RobotInfo.msg uint8 arm_dof: The degrees of freedom of the robotic arm (number of joints) uint8 arm_model: The model of the robotic arm. Examples: 0=RM_65, 1=RM_75, 2=RML_63I (deprecated), 3=RML_63II, 4=RML_63III, 5=ECO_65, 6=ECO_62, 7=GEN_72, 8=ECO63, 9=Generic Robot uint8 force_type: The version of the end-effector force sensor. Examples: 0=Standard, 1=One-Dimensional Force, 2=Six-Dimensional Force, 3=Integrated Six-Dimensional Force uint8 robot_controller_version: The version of the robotic arm controller (3: Third Generation, 4: Fourth Generation) bool state: Whether the reading was successful.
Return example	ros2 topic echo /rm_driver/get_robot_info_result

Get_the_software_information_of_the_robotic_arm

Function description	Get_the_software_information_of_the_robotic_arm
Parameter description	std_msgs::msg::Empty.
Command example	ros2 topic pub --once /rm_driver/get_arm_software_version_cmd std_msgs/msg/Empty "{}"
Return value	Armsoftversion.msg string product_version: The model of the robotic arm string controller_version: The version of the robotic arm controller. If it is a fourth-generation controller, this field will be "4.0" string algorithm_info: Algorithm library information Softwarebuildinfo ctrl_info: Software information for the control layer string dynamic_info: Dynamics version (third generation) Softwarebuildinfo plan_info: Software information for the planning layer (third generation) Softwarebuildinfo com_info: Software information for the communication module (fourth generation) Softwarebuildinfo program_info: Software information for the flowchart programming module (fourth generation) bool state: Query status - true for success, false for failure
Return example	ros2 topic echo /rm_driver/get_arm_software_version_result

Get_the_joint_software_version

Function description	Get the joint software version
Parameter description	std_msgs::msg::Empty.
Command example	ros2 topic pub --once /rm_driver/get_joint_software_version_cmd std_msgs/msg/Empty "{}"
Return value	Jointversion.msg string[] joint_version: An array of software version numbers for each joint obtained. These need to be converted to hexadecimal. For example, if a joint version obtained is 54536, converting it to hexadecimal gives D508, which means the current joint version is Vd5.0.8 (for third-generation controllers). bool state: Acquisition status - true for successful acquisition, false for failed acquisition.
Return example	ros2 topic echo /rm_driver/get_joint_software_version_result

Get_the_software_version_of_the_end_interface_board

Function description	Get_the_software_version_of_the_end_interface_board
Parameter description	std_msgs::msg::Empty.
Command example	ros2 topic pub --once /rm_driver/get_tool_software_version_cmd std_msgs/msg/Empty "{}"
Return value	Toolsoftwareversionv4.msg string tool_version: The software version number of the end-effector interface board bool state: Query status, returns true for success, false for failure
Return example	ros2 topic echo /rm_driver/get_tool_software_version_result

Functions_related_to_the_work_coordinate_system_settings

Change_the_current_work_coordinate_system

Function description	Change_the_current_work_coordinate_system
Parameter description	ROS msg std_msgs::msg::String
Command example	ros2 topic pub /rm_driver/change_work_frame_cmd std_msgs/msg/String "data: 'Base'"
Return value	true-set successfully, false-set failed
Return example	ros2 topic echo /rm_driver/change_work_frame_result

Get_the_current_work_coordinate_system

Function description	Get the current work coordinate system
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub --once /rm_driver/get_curr_workFrame_cmd std_msgs/msg/Empty "{}"

Return value	true-set successfully, false-set failed
Return example	ros2 topic echo /rm_driver/get_curr_workFrame_result

Get_all_work_coordinate_system_names

Function description	Get_all_work_coordinate_system_names
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub --once /rm_driver/get_all_work_frame_cmd std_msgs/msg/Empty "{}"
Return value	All work coordinate system names
Return example	ros2 topic echo /rm_driver/get_all_work_frame_result

Functions_related_to_the_tool_coordinate_system_settings

Change_the_current_tool_coordinate_system

Function description	Change_the_current_tool_coordinate_system
Parameter description	ROS msg std_msgs::msg::String
Command example	ros2 topic pub --once /rm_driver/change_tool_frame_cmd std_msgs/msg/String "data: 'Arm_Tip'"
Return value	true-set successfully, false-set failed
Return example	ros2 topic echo /rm_driver/change_tool_frame_result

Get_the_current_tool_coordinate_system

Function description	Get_the_current_tool_coordinate_system
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub --once /rm_driver/get_current_tool_frame_cmd std_msgs/msg/Empty "{}"
Return value	Current tool coordinate system name
Return example	ros2 topic echo /rm_driver/get_current_tool_frame_result

Get_all_tool_coordinate_system_names

Function description	Get_all_tool_coordinate_system_names
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/get_all_tool_frame_cmd std_msgs/msg/Empty "{}"
Return value	All names of the current tool coordinate system
Return example	ros2 topic echo /rm_driver/get_all_tool_frame_result

Functions_related_to_the_arm_state_query

Get_the_current_state_of_the_robot_arm-return_each_joint_angle_and_Euler_angle

Function description	Retrieve the current state of the robotic
----------------------	---

	arm
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub --once /rm_driver/get_current_arm_state_cmd std_msgs/msg/Empty "{}"
Return value	The current robotic arm joint state (angle) + pose information (Euler angle) + error information
Return example	ros2 topic echo /rm_driver/get_current_arm_original_state_result

Get the current state of the robotic arm-return each joint radians and quaternion

Function description	Retrieve the current state of the robotic arm
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub --once /rm_driver/get_current_arm_state_cmd std_msgs/msg/Empty "{}"
Return value	The current robotic arm joint state (radians) + pose information (quaternion) + error information
Return example	ros2 topic echo /rm_driver/get_current_arm_state_result

Functions related to motion planning of the robotic arm

Joint Space Motion

Function description	Joint space move MOVEJ
Parameter description	Movej.msg float32[6] joint: joint angle, unit: radians. uint8 speed: speed percentage ratio coefficient, 0~100. bool block: whether it is a blocking mode, bool type, true:blocking, false:non-blocking.
Command example	6-degree of freedom ros2 topic pub --once /rm_driver/movej_cmd rm_ros_interfaces/msg/Movej "joint: [0, 0, 0, 0, 0, 0] speed: 20 block: true dof: 6" 7-degree of freedom ros2 topic pub --once /rm_driver/movej_cmd rm_ros_interfaces/msg/Movej "joint: [0, 0, 0, 0, 0, 0] speed: 20 block: true trajectory_connect: 0 dof: 7"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/movej_result

Linear motion in Cartesian space

Function description	Linear motion in Cartesian space MOVEL
Parameter description	Movel.msg geometry_msgs/Pose pose: robotic arm pose, geometry_msgs/Pose type, x, y, z coordinates (float type, unit: m) + quaternion. uint8 speed: speed

	percentage ratio coefficient, 0~100。 bool : whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	First, use MoveJP ros2 topic pub --once /rm_driver/movej_p_cmd rm_ros_interfaces/msg/Movejp "pose: position: x: -0.317239 y: 0.120903 z: 0.255765 orientation: x: -0.983404 y: -0.178432 z: 0.032271 w: 0.006129 speed: 20 block: true" Then use MoveL ros2 topic pub --once /rm_driver/movel_cmd rm_ros_interfaces/msg/MoveL "pose: position: x: -0.317239 y: 0.120903 z: 0.295765 orientation: x: -0.983404 y: -0.178432 z: 0.032271 w: 0.006129 speed: 20 trajectory_connect: 0 block: true"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/movel_result

Cartesian_space_linear_offset_motion

Function description	Cartesian_space_linear_offset_motion
Parameter description	Moveoffset.msg geometriy_msgs/Pose pose: Position and attitude offset, position unit: meters, attitude unit: radians int32 speed: velocity percentage coefficient, 1-100 int32 r: blending radius percentage coefficient, 0-100. Bool trajectory_connect: Trajectory connection flag, 0 immediately plans and executes the trajectory, without connecting to subsequent trajectories. 1: Plan the current trajectory together with the next trajectory, but do not execute it immediately. In blocking mode, even if the transmission is successful, it will immediately return. Bool frame_type: reference coordinate system type, 0 working coordinates, 1 tool coordinates bool block: block setting. In multi-threaded mode, 0 represents non blocking mode, which returns immediately after sending an instruction; 1 represents blocking mode, waiting for the robotic arm to reach the target position or planning failure before returning. In single threaded mode, 0 represents non blocking mode, which returns immediately after sending an instruction; When using other values, block the mode and set a timeout based on the movement time, in seconds.
Command example	First, use MoveJP ros2 topic pub --once /rm_driver/movej_p_cmd rm_ros_interfaces/msg/Movejp "{pose: {position: {x: -0.317239, y: 0.120903, z: 0.255765}, orientation: {x: -0.983404, y: -0.178432, z: 0.032271, w: 0.006129}}, speed: 20, trajectory_connect:

	0, block: true}" Then use Move1_offset ros2 topic pub --once /rm_driver/movel_offset_cmd rm_ros_interfaces/msg/Moveoffset "{pose: {position: {x: -0.317239, y: 0.120903, z: 0.295765}, orientation: {x: -0.983404, y: -0.178432, z: 0.032271, w: 0.006129}}, speed: 20 ,r: 0 ,trajectory_connect: false, frame_type: false,block: false}"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/movel_offset_result

Circular_motion_in_Cartesian_space

Function description	Circular motion in Cartesian space MOVEC
Parameter description	Movec.msg geometry_msgs/Pose pose_mid: middle pose, geometry_msgs/Pose type, x, y, z coordinates (float type, unit: m) + quaternion. geometry_msgs/Pose pose_end: end pose, geometry_msgs/Posetype, x, y, z coordinates (float type, unit: m) + quaternion. uint8 speed: speed percentage ratio coefficient, 0-100. bool block: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	First, use movej_p to reach the specified position ros2 topic pub --once /rm_driver/movej_p_cmd rm_ros_interfaces/msg/Movejp "pose: position: x: 0.274946 y: -0.058786 z: 0.299028 orientation: x: 0.7071 y: -0.7071 z: 0.0 w: 0.0 speed: 0 block: true" Use movec to reach the specified position ros2 topic pub --once /rm_driver/movec_cmd rm_ros_interfaces/msg/Movec "pose_mid: position: x: 0.324946 y: -0.008786 z: 0.299028 orientation: x: 0.7071 y: -0.7071 z: 0.0 w: 0.0 pose_end: position: x: 0.274946 y: 0.041214 z: 0.299028 orientation: x: 0.7071 y: -0.7071 z: 0.0 w: 0.0 speed: 20 trajectory_connect: 0 block: false loop: 0"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/movec_result

Joint_angle_CANFD_transmission

Function description	Joint angle CANFD transmission
Parameter description	Jointpos.msg float32[6] joint: joint angle, unit: radians. bool follow: follow state, true: high following, false: low following, default

	high following if not set. float32 expand: expand joint, unit: radians.
Command example	Transmission needs to send multiple continuous points to achieve, simply by the following command and can not achieve the function, the current moveit2 control using angle transmission control mode. ros2 topic pub /rm_driver/movej_canfd_cmd rm_ros_interfaces/msg/Jointpos "joint: [0, 0, 0, 0, 0, 0] follow: false expand: 0.0 dof: 6"
Return value	Success: no return value; Failure return: the driver terminal returns an error code.

Customize_high_following_mode_joint_angle_CANFD_transmission

Function description	Customize high following mode joint angle CANFD transmission
Parameter description	Jointposcustom.msg float32[6] joint: joint angle, unit: radians. bool follow: follow state, true: high following, false: low following, default high following if not set. float32 expand: expand joint, unit: radians. uint8 trajectory_mode: When the high following mode is set, multiple modes are supported, including 0- complete transparent transmission mode, 1- curve fitting mode and 2- filtering mode. uint8 radio: Set the smoothing coefficient in curve fitting mode (range 0-100) or the filter parameter in filtering mode (range 0-1000). The higher the value, the better the smoothing effect.
Command example	Transmission needs to send multiple continuous points to achieve, simply by the following command and can not achieve the function, the current moveit2 control using angle transmission control mode. ros2 topic pub /rm_driver/movej_canfd_custom_cmd rm_ros_interfaces/msg/Jointposcustom "joint: [0, 0, 0, 0, 0, 0] follow: false expand: 0.0 trajectory_mode: 0 radio: 0 dof: 6"
Return value	Success: no return value; Failure return: the driver terminal returns an error code.

Pose_CANFD_transmission

Function description	Pose CANFD transmission
Parameter description	Cartepos.msg geometry_msgs/Pose pose: transmission pose, geometry_msgs/Pose type, x, y, z coordinates (float type, unit: m) + quaternion. bool follows: follow state, true: high following, false: low following, default high following if not set.
Command example	It needs to be a large number (10 or more) of continuous position points, simply by the following command and can not achieve the function, with more than a 2ms period

	continuous release. ros2 topic pub /rm_driver/movep_canfd_cmd rm_ros_interfaces/msg/Cartepos "pose: position: x: 0.0 y: 0.0 z: 0.0 orientation: x: 0.0 y: 0.0 z: 0.0 w: 1.0 follow: false"
Return value	Success: no return value; Failure return: the driver terminal returns an error code.

Customize_high_following_mode_pose_CANFD_transmission

Function description	Customize high following mode pose CANFD transmission
Parameter description	Carteposcustom.msg geometry_msgs/Pose pose: transmission pose, geometry_msgs/Pose type, x, y, z coordinates (float type, unit: m) + quaternion. bool follows: follow state, true: high following, false: low following, default high following if not set. uint8 trajectory_mode: When the high following mode is set, multiple modes are supported, including 0- complete transparent transmission mode, 1- curve fitting mode and 2- filtering mode. uint8 radio: Set the smoothing coefficient in curve fitting mode (range 0-100) or the filter parameter in filtering mode (range 0-1000). The higher the value, the better the smoothing effect.
Command example	It needs to be a large number (10 or more) of continuous position points, simply by the following command and can not achieve the function, with more than a 2ms period continuous release. ros2 topic pub /rm_driver/movep_canfd_custom_cmd rm_ros_interfaces/msg/Carteposcustom "pose: position: x: 0.0 y: 0.0 z: 0.0 orientation: x: 0.0 y: 0.0 z: 0.0 w: 1.0 follow: false trajectory_mode: 0 radio: 0"
Return value	Success: no return value; Failure return: the driver terminal returns an error code.

Joint_space_planning_to_target_pose

Function description	Joint space planning to target pose MOVEJP
Parameter description	Movejp.msg geometry_msgs/Pose target pose, x, y, z coordinates (float type, unit: m) + quaternion. uint8 speed: speed percentage ratio coefficient, 0-100. bool block: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	ros2 topic pub --once /rm_driver/movej_p_cmd rm_ros_interfaces/msg/Movejp "pose: position: x: -0.317239 y: 0.120903 z: 0.255765 orientation: x: -0.983404 y: -0.178432 z: 0.032271 w: 0.006129"

	speed: 20 block: true"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/movej_p_result

Trajectory_emergency_stop

Function description	Motion planning trajectory emergency stop
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/move_stop_cmd std_msgs/msg/Empty "{}"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/move_stop_result

Emergency_stop

Function description	Emergency_stop
Parameter description	rm_ros_interfaces/Stop Emergency stop status, true: E-stopped, false: Resumed
Command example	ros2 topic pub --once /rm_driver/emergency_stop_cmd rm_ros_interfaces/Stop "state: true"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/emergency_stop_result

Emergency_pause

Function description	Emergency pause
Parameter description	std_msgs::msg::Empty
Command example	ros2 topic pub --once /rm_driver/pause_cmd std_msgs/msg/Empty "{}"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/pause_result

Resume_after_trajectory_pause

Function description	Resume after trajectory pause
Parameter description	std_msgs::msg::Empty
Command example	ros2 topic pub --once /rm_driver/set_arm_continue_cmd std_msgs/msg/Empty "{}"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_arm_continue_result

Teaching_instructions

Joint_teaching

Function description	Joint teaching
Parameter description	Jointteach.msg uint8 num:Joint num, 1~7 uint8 direction:teach direction, 0-negative direction, 1-positive direction uint8 speed:speed percentage ratio coefficient, 0-100.
Command example	ros2 topic pub /rm_driver/set_joint_teach_cmd rm_ros_interfaces/msg/Jointteach "num: 1 direction: 0 speed: 10"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_joint_teach_result

Position_teaching

Function description	Position teaching
Parameter description	Posteach.msg uint8 type: Teaching demonstration type: input0:X-axis direction、1:Y-axis direction、2:Z-axis direction uint8 direction:teach direction, 0- negative direction, 1-positive direction uint8 speed:speed percentage ratio coefficient, 0-100.
Command example	ros2 topic pub /rm_driver/set_pos_teach_cmd rm_ros_interfaces/msg/Posteach "type: 2 direction: 0 speed: 10"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_pos_teach_result

Attitude_teaching

Function description	Attitude teaching
Parameter description	Ortteach.msg uint8 type: Teaching demonstration type: input0:RX-axis direction、1:RY-axis direction、2:RZ-axis direction uint8 direction:teach direction, 0- negative direction, 1-positive direction uint8 speed:speed percentage ratio coefficient, 0-100.
Command example	ros2 topic pub /rm_driver/set_ort_teach_cmd rm_ros_interfaces/msg/Ortteach "type: 2 direction: 0 speed: 10"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_ort_teach_result

Stop_teaching

Function description	Stop teaching
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/set_stop_teach_cmd std_msgs/msg/Empty "{}"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_stop_teach_result

Trajectory_File

Query_Trajectory_List

Function description	Query_Trajectory_List
Parameter description	Gettrajjectorylist.msg int32 page_num: Page number. Int32 page_size: The size of each page. String vague_search: Fuzzy search.
Command example	ros2 topic pub --once /rm_driver/get_trajectory_file_list_cmd rm_ros_interfaces/msg/Gettrajjectorylist "{page_num: 1,page_size: 10,vague_search: 's'}"
Return value	Trajectorylist.msg int32 page_num # Page number int32 page_size # Size per page int32 total_size # Length of the list

string vague_search # Fuzzy search Trajectoryinfo[] tra_list # List of trajectories that match the criteria bool state # Query status - true for success, false for failure|

Return example	ros2 topic echo /rm_driver/get_trajectory_file_list_result
----------------	---

Start_Running_Specified_Trajectory

Function description	Start_Running_Specified_Trajectory
Parameter description	ROS msg std_msgs::msg::String
Command example	ros2 topic pub --once /rm_driver/set_run_trajectory_cmd std_msgs/msg/String "data: 'sss'"
Return value	Successfully returned query trajectory information; Failed with no return, the driver terminal returned an error code.
Return example	ros2 topic echo /rm_driver/set_run_trajectory_result

Delete_Specified_Trajectory_File

Function description	Delete_Specified_Trajectory_File
Parameter description	ROS msg std_msgs::msg::String
Command example	ros2 topic pub --once /rm_driver/delete_trajectory_file_cmd

	std_msgs/msg/String "data: 'sss'"
Return value	Successfully returned query trajectory information; Failed with no return, the driver terminal returned an error code.
Return example	ros2 topic echo /rm_driver/delete_trajectory_file_result

Save_Trajectory_File

Function description	Save_Trajectory_File
Parameter description	ROS msg std_msgs::msg::String
Command example	ros2 topic pub --once /rm_driver/save_trajectory_file_cmd std_msgs/msg/String "data: 'test'"
Return value	Successfully returned query trajectory information; Failed with no return, the driver terminal returned an error code.
Return example	ros2 topic echo /rm_driver/save_trajectory_file_result

Query_Flowchart_Program_Run_State

Function description	Query_Flowchart_Program_Run_State
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub --once /rm_driver/get_flowchart_program_run_state_cmd std_msgs/msg/Empty "{}"
Return value	Flowchart_runstate.msg int run_state; #Running Status 0 Not Started 1 Running 2 Paused int id;#The current enabled file ID. char name[32];#The name of the currently enabled file. int plan_speed;#The current enabled file global planning speed ratio is 1-100. int step_mode;#Single step mode, 0 is empty, 1 is normal, and 2 is single step. char modal_id[50];#The ID of the flowchart block that has been run. If it has not been run, no return will be sent.
Return example	ros2 topic echo /rm_driver/get_flowchart_program_run_state_result

Modbus_Configuration

Set_Controller_RS485_Mode

Function description	Set_Controller_RS485_Mode
Parameter description	RS485params.msg int32 mode: 0-RS485 serial communication, 1-modbus RTU master mode, 2-Modbus RTU slave mode. Int32 baudrate: Currently supports 9600 19200 38400 57600 115200 230400 460800.
Command example	ros2 topic pub /rm_driver/set_controller_rs485_mode_cmd rm_ros_interfaces/msg/RS485params "{mode: 0, baudrate: 115200}"
Return value	Successful return: true; failure returns:

	false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_controller_rs485_mode_result

Get_Controller_RS485_Mode

Function description	Get_Controller_RS485_Mode
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/get_controller_rs485_mode_cmd std_msgs/msg/Empty "{}"
Return value	RS485params.msg int32 mode: 0-RS485 serial communication, 1-modbus RTU master mode, 2-Modbus RTU slave mode. Int32 baudrate: Currently supports 9600 19200 38400 57600 115200 230400 460800.
Return example	ros2 topic echo /rm_driver/get_controller_rs485_mode_result

Set_Tool_End_RS485_Mode

Function description	Set_Tool_End_RS485_Mode
Parameter description	RS485params.msg int32 mode: 0- Set the RS485 port of the tool end to RTU master station, 1- Smart hand mode, 2- Claw mode. Int32 baudrate: Currently supports 9600 115200 460800.
Command example	ros2 topic pub --once /rm_driver/set_tool_rs485_mode_cmd rm_ros_interfaces/msg/RS485params "{mode: 0, baudrate: 115200}"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_tool_rs485_mode_result

Get_Tool_End_RS485_Mode

Function description	Get_Tool_End_RS485_Mode
Parameter description	ROS msg std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/get_tool_rs485_mode_cmd std_msgs/msg/Empty "{}"
Return value	RS485params.msg int32 mode: 0- Set the RS485 port of the tool end to RTU master station, 1- Smart hand mode, 2- Claw mode. Int32 baudrate: Currently supports 9600 115200 460800.
Return example	ros2 topic echo /rm_driver/get_tool_rs485_mode_result

Set_Controller_And_Tools_ModbusRTU_Mode-Three_Generations

Function description	Set_Controller_And_Tools_ModbusRTU_Mode
Parameter description	RS485params.msg int32 mode: 0- Set the RS485 port of the tool end to RTU master station, 1- Smart hand mode, 2- Claw mode. Int32 baudrate: Currently supports 9600 115200 460800.
Command example	ros2 topic pub --once /rm_driver/set_controller_rs485_mode_cmd rm_ros_interfaces/msg/RS485params "{mode: 0, baudrate: 115200, state: false}"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/get_tool_rs485_mode_result

Close_Controller_And_Tools_ModbusRTU_Mode-Three_Generations

Function description	Close_Controller_And_Tools_ModbusRTU_Mode
Parameter description	std_msgs::msg::UInt16 UInt16 data: 0- Close the RS485 port of the tool end to RTU master station, 1- Smart hand mode, 2- Claw mode.
Command example	ros2 topic pub --once /rm_driver/close_controller_rtu_modbus_cmd std_msgs/msg/UInt16 "data: 0"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/get_tool_rs485_mode_result

ModbusTCP_Master

Add_Modbus_TCP_Master

Function description	Add_Modbus_TCP_Master
Parameter description	Modbus tcp master info.msg string master_name: Modbus master station name. String ip: TCP master IP address. Int32 port: TCP primary port number.
Command example	ros2 topic pub /rm_driver/add_modbus_tcp_master_cmd rm_ros_interfaces/msg/Modbus tcp master info "{master_name: '1', ip: '127.0.0.1', port: 502}"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/add_modbus_tcp_master_result

Update_Modbus_TCP_Master

Function description	Update_Modbus_TCP_Master
Parameter description	Modbustcpmasterupdate.msg string master_name: Modbus old master station name. string new_master_name: Modbus new master station name. String ip: TCP master IP address. Int32 port: TCP primary port number.
Command example	ros2 topic pub /rm_driver/update_modbus_tcp_master_cmd rm_ros_interfaces/msg/Modbustcpmasterupdate '{"master_name: '1',new_master_name: '125',ip: '127.0.0.1',port: 502}'"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/update_modbus_tcp_master_result

Delete_Modbus_TCP_Master

Function description	Delete_Modbus_TCP_Master
Parameter description	Mastername.msg string master_name: Modbus master station name.
Command example	ros2 topic pub /rm_driver/delete_modbus_tcp_master_cmd rm_ros_interfaces/msg/Mastername "master_name: '321'"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/delete_modbus_tcp_master_result

Get_Specified_Modbus_TCP_Master

Function description	Get_Specified_Modbus_TCP_Master
Parameter description	Mastername.msg string master_name: Modbus master station name.
Command example	ros2 topic pub /rm_driver/get_modbus_tcp_master_cmd rm_ros_interfaces/msg/Mastername "master_name: '321'"
Return value	Modbustcpmasterinfo.msg string master_name # Name of the Modbus master station, with a maximum length of 15 characters, not exceeding 15 characters string ip # IP address of the TCP master station int32 port # Port number of the TCP master station bool state # Query status information, false for failure, true for success On failure, the driver terminal returns an error code.

Return example	ros2 topic echo /rm_driver/get_modbus_tcp_master_result
----------------	--

Get_Modbus_TCP_Master_List

Function description	Get_Modbus_TCP_Master_List
Parameter description	Getmodbustcpmasterlist.msg int32 page_num: Page number. Int32 page_size: The size of each page. String vague_search: Fuzzy search, returns a list of all main sites if the input is empty.
Command example	ros2 topic pub /rm_driver/get_modbus_tcp_master_list_cm d rm_ros_interfaces/msg/Getmodbustcpmast erlist "{page_num: 1,page_size: 10,vague_search: '1'}"
Return value	Modbustcpmasterlist.msg uint8 page_num # Page number uint8 page_size # Size per page uint8 total_size # Length of the list string vague_search # Fuzzy search Modbustcpmasterinfo[] master_list # List of TCP master stations that match the criteria bool state # Query status - true for success, false for failure On failure, the driver terminal returns an error code
Return example	ros2 topic echo /rm_driver/get_modbus_tcp_master_list_res ult

Set_Modbus_TCP_Master-Three_Generations

Function description	Add_Modbus_TCP_Master
Parameter description	Modbustcpmasterinfo.msg string master_name: Modbus master station name(No configuration required). String ip: TCP master IP address. Int32 port: TCP primary port number.
Command example	ros2 topic pub --once /rm_driver/set_controller_tcp_mode_cmd rm_ros_interfaces/msg/Modbustcpmasterinf o "{master_name: '', ip: '192.168.1.18', port: 502, state: false}"
Return value	Successful return: true; failure returns: false, and the driver terminal returns an error code.
Return example	ros2 topic echo rm_driver/set_controller_tcp_mode_result

Close_Modbus_TCP_Master-Three_Generations

Function description	Close_Modbus_TCP_Master
Parameter description	Empty.msg.
Command example	ros2 topic pub --once /rm_driver/close_controller_tcp_modbus_cm d std_msgs/msg/Empty "{}"
Return value	Successful return: true; failure returns:

	false, and the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/close_controller_tcp_modbus_result

Tool_end_controller_end_RTU_Modbus_protocol_reads_and_writes_data

Modbus_RTU_Protocol_Read_Coils-Four_Generations

Function description	Modbus_RTU_Protocol_Read_Coils
Parameter description	Modbusrtureadparams.msg int32 address: data starting address. Int32 device: Peripheral device address. int32 type: 0- Controller end Modbus host; 1. Tool end Modbus host. Int32 num: The number of data to be read, with a length not exceeding 100.
Command example	ros2 topic pub /rm_driver/read_modbus_rtu_coils_cmd rm_ros_interfaces/msg/Modbusrtureadparams "{address: 0, device: 0, type: 0, num: 1}"
Return value	Modbusreaddata.msg int32[] read_data # Data read from Modbus bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_rtu_coils_result

Modbus_RTU_Protocol_Write_Coils-Four_Generations

Function description	Modbus_RTU_Protocol_Write_Coils
Parameter description	Modbusrtuwriteparams.msg int32 address: data starting address. Int32 device: Address of peripheral device. Int32 type: 0- Controller side Modbus host; 1. Tool side Modbus host. Int32 num: The maximum number of data to be written is 100. Int32 [] data: The data to be written, with a length corresponding to num.
Command example	ros2 topic pub /rm_driver/write_modbus_rtu_coils_cmd rm_ros_interfaces/msg/Modbusrtuwriteparams "{address: 0, device: 0, type: 0, num: 2, data: [1,1]}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/write_modbus_rtu_coils_result

Modbus_RTU_Protocol_Read_Discrete_Inputs-Four_Generations

Function description	Modbus_RTU_Protocol_Read_Discrete_Inputs
Parameter description	Modbusrtureadparams.msg int32 address:

	data starting address. Int32 device: Peripheral device address. int32 type: 0-Controller side Modbus host; 1. Tool side Modbus host. Int32 num: The number of data to be read, with a data length not exceeding 100.
Command example	ros2 topic pub /rm_driver/read_modbus_rtu_input_status_cmd rm_ros_interfaces/msg/Modbusrtureadparams "{address: 0, device: 0, type: 0, num: 1}"
Return value	Modbusreaddata.msg int32[] read_data # Data read from Modbus bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_rtu_input_status_result

Modbus_RTU_Protocol_Read_Holding_Registers-Four_Generations

Function description	Modbus_RTU_Protocol_Read_Discrete_Inputs
Parameter description	Modbusrtureadparams.msg int32 address: data starting address. Int32 device: Peripheral device address. int32 type: 0-Controller end Modbus host; 1. Tool end Modbus host. Int32 num: The number of data to be read, with a data length not exceeding 100.
Command example	ros2 topic pub /rm_driver/read_modbus_rtu_holding_registers_cmd rm_ros_interfaces/msg/Modbusrtureadparams "{address: 0, device: 0, type: 0, num: 1}"
Return value	Modbusreaddata.msg int32[] read_data # The data read from Modbus bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_rtu_holding_registers_result

Modbus_RTU_Protocol_Write_Holding_Registers-Four_Generations

Function description	Modbus_RTU_Protocol_Write_Holding_Registers
Parameter description	Modbusrtuwritableparams.msg int32 address: data starting address int32 device: peripheral device address int32 type: 0-controller end Modbus host; 1. Tool end Modbus host. Int32 num: The maximum amount of data to be written, not exceeding 100 int32 [] data: The data to be

	written, with a length corresponding to num.
Command example	ros2 topic pub /rm_driver/write_modbus_rtu_registers_cmd rm_ros_interfaces/msg/Modbusrtuwritepara ms "{address: 0, device: 0, type: 0, num: 2, data: [1,1]}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/write_modbus_rtu_registers_resu lt

Modbus_RTU_Protocol_Read_Input_Registers-Four_Generations

Function description	Modbus_RTU_Protocol_Read_Input_Register s
Parameter description	Modbusrtureadparams.msg int32 address: data starting address. Int32 device: Peripheral device address. int32 type: 0- Controller side Modbus host; 1. Tool side Modbus host.. Int32 num: The number of data to be read, with a data length not exceeding 100.
Command example	ros2 topic pub /rm_driver/read_modbus_rtu_input_register s_cmd rm_ros_interfaces/msg/Modbusrtureadpara ms "{address: 0, device: 0, type: 0, num: 1}"
Return value	Modbusreaddata.msg int32[] read_data # Data read from Modbus bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_rtu_input_register s_result

Modbus_RTU_Protocol_Read_Coils-Three_Generations

Function description	Modbus_RTU_Protocol_Read_Coils
Parameter description	Modbusrtureadparams.msg int32 address: data starting address. Int32 device: Peripheral device address. int32 type: 0- Controller end Modbus host; 1. Tool end Modbus host. Int32 num: The number of data to be read, with a length not exceeding 120.
Command example	ros2 topic pub --once /rm_driver/read_modbus_rtu_coils_cmd rm_ros_interfaces/msg/Modbusrtureadpara ms "{address: 0, device: 1, type: 0, num: 1}"
Return value	Modbusreaddata.msg int32[] read_data # Data read from Modbus, (When reading, 8- bit data will be combined into a byte and

	returned). bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_rtu_coils_result

Modbus_RTU_Protocol_Write_Coils-Three_Generations

Function description	Modbus_RTU_Protocol_Write_Coils
Parameter description	Modbusrtuwriteparams.msg int32 address: data starting address. Int32 device: Address of peripheral device. Int32 type: 0- Controller side Modbus host; 1. Tool side Modbus host. Int32 num: The maximum number of data to be written is 160. Int32 [] data: The data to be written, with a length corresponding to num(When writing, it is necessary to combine 8-bit data into a byte for writing).
Command example	ros2 topic pub --once /rm_driver/write_modbus_rtu_coils_cmd rm_ros_interfaces/msg/Modbusrtuwriteparams "{address: 0, device: 1, type: 0, num: 8, data: [3]}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/write_modbus_rtu_coils_result

Modbus_RTU_Protocol_Read_Discrete_Inputs-Three_Generations

Function description	Modbus_RTU_Protocol_Read_Discrete_Inputs
Parameter description	Modbusrtureadparams.msg int32 address: data starting address. Int32 device: Peripheral device address. int32 type: 0- Controller side Modbus host; 1. Tool side Modbus host. Int32 num: The number of data to be read, with a data length not exceeding 8.
Command example	ros2 topic pub --once /rm_driver/read_modbus_rtu_input_status_cmd rm_ros_interfaces/msg/Modbusrtureadparams "{address: 0, device: 2, type: 0, num: 1}"
Return value	Modbusreaddata.msg int32[] read_data # Data read from Modbus,(When reading, 8-bit data will be combined into a byte and returned). bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_rtu_input_status_result

Modbus_RTU_Protocol_Read_Holding_Registers-Three_Generations

Function description	Modbus_RTU_Protocol_Read_Discrete_Inputs
Parameter description	Modbusrtureadparams.msg int32 address: data starting address. Int32 device: Peripheral device address. int32 type: 0-Controller end Modbus host; 1. Tool end Modbus host. Int32 num: The number of data to be read, with a data length not exceeding 12.
Command example	ros2 topic pub --once /rm_driver/read_modbus_rtu_holding_registers_cmd rm_ros_interfaces/msg/Modbusrtureadparams "{address: 0, device: 2, type: 0, num: 1}"
Return value	Modbusreaddata.msg int32[] read_data # The data read from Modbus.This instruction supports reading up to 12 registers of data at a time, which amounts to 24 bytes (each data is represented as a 16-bit value, with the rest displayed in two-digit octal notation). bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_rtu_holding_registers_result

Modbus_RTU_Protocol_Write_Holding_Registers-Three_Generations

Function description	Modbus_RTU_Protocol_Write_Holding_Registers
Parameter description	Modbusrtuwritableparams.msg int32 address: data starting address int32 device: peripheral device address int32 type: 0-controller end Modbus host; 1. Tool end Modbus host. Int32 num: The maximum amount of data to be written, not exceeding 10 int32 [] data: The data to be written, with a length corresponding to num,The maximum size should not exceed 10 (when writing multiple items, it is necessary to split a 16-bit value into two 8-bit values for writing).
Command example	ros2 topic pub --once /rm_driver/write_modbus_rtu_registers_cmd rm_ros_interfaces/msg/Modbusrtuwritableparams "{address: 0, device: 2, type: 0, num: 2, data: [1,1,2,3]}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/write_modbus_rtu_registers_result

	It
--	----

Modbus_RTU_Protocol_Read_Input_Registers-Three_Generations

Function description	Modbus_RTU_Protocol_Read_Input_Registers
Parameter description	Modbusrtureadparams.msg int32 address: data starting address. Int32 device: Peripheral device address. int32 type: 0- Controller side Modbus host; 1. Tool side Modbus host.. Int32 num: The number of data to be read, with a data length not exceeding 12.
Command example	ros2 topic pub --once /rm_driver/read_modbus_rtu_input_registers_cmd rm_ros_interfaces/msg/Modbusrtureadparams "{address: 0, device: 2, type: 0, num: 1}"
Return value	Modbusreaddata.msg int32[] read_data # Data read from Modbus.The data length should not exceed 12. This instruction supports reading up to 12 register data at a time, which is 24 bytes (when the number of reads exceeds 1, the data will be split into two 8-bit returns). bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_rtu_input_registers_result

Controller_Modbus_TCP_protocol_reads_and_writes_data

Modbus_TCP_Protocol_Read_Coils-Four_Generations

Function description	Modbus_TCP_Protocol_Read_Coils
Parameter description	Modbustcpreadparams.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters. String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Read data quantity, maximum not exceeding 100.
Command example	ros2 topic pub /rm_driver/read_modbus_tcp_coils_cmd rm_ros_interfaces/msg/Modbustcpreadparams "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 1}"
Return value	Modbusreaddata.msg int32[] read_data # Data read from Modbus bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_tcp_coils_result

Modbus_TCP_Protocol_Write_Coils-Four_Generations

Function description	Modbus_TCP_Protocol_Write_Coils
Parameter description	ModbusTCPWriteParams.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters. String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Write data quantity, maximum not exceeding 100. Int32 [] data: The written data has a length corresponding to num.
Command example	ros2 topic pub /rm_driver/write_modbus_tcp_coils_cmd rm_ros_interfaces/msg/ModbusTCPWriteParams "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 2, data: [1,1]}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/write_modbus_tcp_coils_result

Modbus_TCP_Protocol_Read_Discrete_Inputs-Four_Generations

Function description	Modbus_TCP_Protocol_Read_Discrete_Inputs
Parameter description	ModbusTCPReadParams.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters. String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Read data quantity, maximum not exceeding 100.
Command example	ros2 topic pub /rm_driver/read_modbus_tcp_input_status_cmd rm_ros_interfaces/msg/ModbusTCPReadParams "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 1}"
Return value	ModbusReadData.msg int32[] read_data # Data read from Modbus bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_tcp_input_status_result

Modbus_TCP_Protocol_Read_Holding_Registers-Four_Generations

Function description	Modbus_TCP_Protocol_Read_Holding_Registers
Parameter description	ModbusTCPReadParams.msg int32 address:

	data starting address. String master_name: Modbus master name, maximum length of 15 characters. String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Read data quantity, maximum not exceeding 100.
Command example	ros2 topic pub /rm_driver/read_modbus_tcp_holding_registers_cmd rm_ros_interfaces/msg/ModbusTCPReadParams "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 1}"
Return value	ModbusReadData.msg int32[] read_data # Data read from Modbus bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_tcp_holding_registers_result

Modbus_TCP_Protocol_Write_Holding_Registers-Four_Generations

Function description	Modbus_TCP_Protocol_Write_Holding_Registers
Parameter description	ModbusTCPWriteParams.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters. String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Write data quantity, maximum not exceeding 100. Int32 [] data: The written data has a length corresponding to num.
Command example	ros2 topic pub /rm_driver/write_modbus_tcp_registers_cmd rm_ros_interfaces/msg/ModbusTCPWriteParams "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 2, data: [1,1]}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/write_modbus_tcp_registers_result

Modbus_TCP_Protocol_Read_Input_Registers-Four_Generations

Function description	Modbus_TCP_Protocol_Read_Input_Registers
Parameter description	ModbusTCPReadParams.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters. String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num:

	Read data quantity, maximum not exceeding 100.
Command example	ros2 topic pub /rm_driver/read_modbus_tcp_input_registers_cmd rm_ros_interfaces/msg/ModbusTCPReadParameters "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 1}"
Return value	ModbusReadData.msg int32[] read_data # Data read from Modbus bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_tcp_input_registers_result

Modbus_TCP_Protocol_Read_Coils-Three_Generations

Function description	Modbus_TCP_Protocol_Read_Coils
Parameter description	ModbusTCPReadParameters.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters(No configuration required). String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Read data quantity, maximum not exceeding 120.
Command example	ros2 topic pub --once /rm_driver/read_modbus_tcp_coils_cmd rm_ros_interfaces/msg/ModbusTCPReadParameters "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 1}"
Return value	ModbusReadData.msg int32[] read_data # Data read from Modbus, (When reading, 8-bit data will be combined into a byte and returned). bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_rtu_coils_result

Modbus_TCP_Protocol_Write_Coils-Three_Generations

Function description	Modbus_TCP_Protocol_Write_Coils
Parameter description	ModbusTCPWriteParameters.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters(No configuration required). String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Write data quantity, maximum not exceeding 160. Int32 [] data: The written data has a length corresponding to num(When writing, it is necessary to combine 8-bit data into a byte for writing).

Command example	<code>ros2 topic pub --once /rm_driver/write_modbus_tcp_coils_cmd rm_ros_interfaces/msg/ModbusTcpWriteParameters "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 10, data: [2,3]}"</code>
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	<code>ros2 topic echo /rm_driver/write_modbus_tcp_coils_result</code>

Modbus_TCP_Protocol_Read_Discrete_Inputs-Three_Generations

Function description	Modbus_TCP_Protocol_Read_Discrete_Inputs
Parameter description	ModbusTcpReadParameters.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters(No configuration required). String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Read data quantity, maximum not exceeding 8.
Command example	<code>ros2 topic pub --once /rm_driver/read_modbus_tcp_input_status_cmd rm_ros_interfaces/msg/ModbusTcpReadParameters "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 1}"</code>
Return value	ModbusReadData.msg int32[] read_data # Data read from Modbus,(When reading, 8-bit data will be combined into a byte and returned). bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	<code>ros2 topic echo /rm_driver/read_modbus_tcp_input_status_result</code>

Modbus_TCP_Protocol_Read_Holding_Registers-Three_Generations

Function description	Modbus_TCP_Protocol_Read_Holding_Registers
Parameter description	ModbusTcpReadParameters.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters(No configuration required). String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Read data quantity, maximum not exceeding 12.
Command example	<code>ros2 topic pub /rm_driver/read_modbus_tcp_holding_registers_cmd rm_ros_interfaces/msg/ModbusTcpReadParameters "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 1}"</code>

Return value	Modbusreaddata.msg int32[] read_data # Data read from Modbus.This instruction supports reading up to 12 registers of data at a time, which amounts to 24 bytes (each data is represented as a 16-bit value, with the rest displayed in two-digit octal notation). bool state # Feedback query status information, false for failure, true for success On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_tcp_holding_registers_result

Modbus_TCP_Protocol_Write_Holding_Registers-Three_Generations

Function description	Modbus_TCP_Protocol_Write_Holding_Registers
Parameter description	Modbus tcpwriteparams.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters(No configuration required). String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Write data quantity, maximum not exceeding 10. Int32 [] data: The written data has a length corresponding to num.The maximum size should not exceed 10 (when writing multiple items, it is necessary to split a 16-bit value into two 8-bit values for writing).
Command example	When writing to multiple registers, it is necessary to split the register contents into high and low bits. For example, to write 257, you should enter 1, 1 ros2 topic pub --once /rm_driver/write_modbus_tcp_registers_cmd rm_ros_interfaces/msg/Modbus tcpwriteparams "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 2, data: [100,100,100,200]}" Write a single register by directly inputting the corresponding value ros2 topic pub --once /rm_driver/write_modbus_tcp_registers_cmd rm_ros_interfaces/msg/Modbus tcpwriteparams "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 1, data: [1000]}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/write_modbus_tcp_registers_result

Modbus_TCP_Protocol_Read_Input_Registers-Three_Generations

Function description	Modbus_TCP_Protocol_Read_Input_Registers
Parameter description	ModbusTCPreadparams.msg int32 address: data starting address. String master_name: Modbus master name, maximum length of 15 characters. String ip: The IP address of the host connection. Int32 port: The port number of the host connection. Int32 num: Read data quantity, maximum not exceeding 12.
Command example	ros2 topic pub --once /rm_driver/read_modbus_tcp_input_registers_cmd rm_ros_interfaces/msg/ModbusTCPreadparams "{address: 0, master_name: '3', ip: '127.0.0.6', port: 502, num: 1}"
Return value	Modbusreaddata.msg int32[] read_data # Data read from Modbus. The data length should not exceed 12. This instruction supports reading up to 12 register data at a time, which is 24 bytes (when the number of reads exceeds 1, the data will be split into two 8-bit returns). bool state # Feedback query status information, false for failure, true for success. On failure, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/read_modbus_tcp_input_registers_result

Functions_related_to_the_IO_configuration_of_the_end_tool

Setting_the_tool_voltage_output

Function description	Setting the tool voltage output
Parameter description	ROS msg: std_msgs::msg::UInt16 uint16 data: power output type, range: 0~3 0-0V, 1-5V, 2-12V, 3-24V
Command example	ros2 topic pub --once /rm_driver/set_tool_voltage_cmd std_msgs/msg/UInt16 "data: 0"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_tool_voltage_result

Functions_related_to_the_control_of_the_end_gripper

The RealMan robotic arm is equipped with an Inspire Robots EG2-4C2 gripper. The robotic arm controller has made the gripper's ROS control mode available to the user to facilitate user operation.

Setting_the_Gripper_Pick

Function description	Setting the gripper pick
Parameter description	Gripperpick.msg uint16 speed: 1~1000, representing the opening and closing

	speed of the gripper, dimensionless. uint16 force: representing the gripping force of the gripper, maximum 1.5 kg. bool block: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	ros2 topic pub --once /rm_driver/set_gripper_pick_cmd rm_ros_interfaces/msg/Gripperpick "speed: 200 force: 200 block: true timeout: 1000"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_gripper_pick_result

Setting_the_gripper_pick-on

Function description	Setting the gripper pick-on
Parameter description	Gripperpick.msg uint16 speed: 1~1000, representing the opening and closing speed of the gripper, dimensionless. uint16 force: 1~1000, representing the gripping force of the gripper, maximum 1.5 kg. bool block: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	ros2 topic pub --once /rm_driver/set_gripper_pick_on_cmd rm_ros_interfaces/msg/Gripperpick "speed: 200 force: 200 block: true timeout: 1000"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_gripper_pick_on_result

Setting_the_gripper_to_the_given_position

Function description	Setting the gripper to the given position
Parameter description	Gripperset.msg uint16 position: target position of the gripper, range: 1-1000, representing the opening degree of the gripper: 0-70 mm. bool block: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	ros2 topic pub --once /rm_driver/set_gripper_position_cmd rm_ros_interfaces/msg/Gripperset "position: 500 block: true timeout: 1000"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_gripper_position_result

Functions_related_to_the_drag_teach_and_trajectory_reproduction

Set_the_force-position_mixing_control

Function description	Set the force-position mixing control
Parameter description	Setforceposition.msg uint8 sensor: 0 - One-axis force; 1 - Six-axis force uint8 mode: 0 - Base coordinate system force control; 1 - Tool coordinate system force control uint8 direction: Force control direction; 0 - Along the X-axis; 1 - Along the Y-axis; 2 - Along the Z-axis; 3 - Along the RX posture direction; 4 - Along the RY posture direction; 5 - Along the RZ posture direction int16 n: The value of force, unit: N, accuracy: 0.1N
Command example	ros2 topic pub --once /rm_driver/set_force_postion_cmd rm_ros_interfaces/msg/Setforceposition "sensor: 1 mode: 0 direction: 2 n: 3"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_force_postion_result

Stop_the_force-position_mixing_control

Function description	Stop the force-position mixing control
Parameter description	std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/stop_force_postion_cmd std_msgs/msg/Empty "{}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/clear_force_data_result

Functions_related_to_the_use_of_six-axis_force_sensors_at_the_end

The RealMan RM-65F robotic arm has an integrated six-axis force sensor at the end without external wiring. Users can operate the six-axis force through ROS topics.

Query_the_six-axis_force_data

Function description	Query the six-axis force data
Parameter description	std_msgs::msg::Empty
Command example	ros2 topic pub rm_driver/get_force_data_cmd std_msgs/msg/Empty "{}"
Return value	Successfully returned six-axis force data in the corresponding coordinate system.
Return example	ros2 topic echo /rm_driver/get_force_data_result displays the raw force data. ros2 topic echo /rm_driver/get_zero_force_data_result displays the system force data. ros2 topic echo /rm_driver/get_work_force_data_result displays the force data in the work

	coordinate system. ros2 topic echo /rm_driver/get_tool_force_data_result displays the force data in the tool coordinate system.
--	---

Clearing_the_six-axis_force_Data

Function description	Clearing the six-axis force data
Parameter description	std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/clear_force_data_cmd std_msgs/msg/Empty "{}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/clear_force_data_result

Functions_related_to_the_control_of_the_five-finger_dexterous_hand

The RealMan RM-65 robotic arm has been equipped with a five-finger dexterous hand at the end. Users can set the hand through the ROS.

Setting_the_serial_number_of_the_dexterous_hand_posture

Function description	Setting the serial number of the dexterous hand posture
Parameter description	Handposture.msg uint16 posture_num: the serial number of the posture pre-saved in the dexterous hand, ranging from 1 to 40. bool data: whether it is a blocking mode, bool type, true: blocking, false: non-blocking. uint16 timeout: The timeout setting for blocking mode, unit: seconds.
Command example	ros2 topic pub --once /rm_driver/set_hand_posture_cmd rm_ros_interfaces/msg/Handposture "posture_num: 1 block: true timeout: 1000"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_hand_posture_result

Set_the_dexterous_hand_action_sequence_number

Function description	Set the dexterous hand action sequence number
Parameter description	Handseq.msg uint16 seq_num: the serial number of the action sequence pre-saved in the dexterous hand, ranging from 1 to 40. bool data: whether it is a blocking mode, bool type, true: blocking, false: non-blocking. uint16 timeout: The timeout setting for blocking mode, unit: seconds.
Command example	ros2 topic pub --once /rm_driver/set_hand_seq_cmd rm_ros_interfaces/msg/Handseq "seq_num:

	1 block: true timeout:1000"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_hand_seq_result

Setting_the_angles_of_various_degrees_of_freedom_for_the_dexterous_hand

Function description	Setting the angles of various degrees of freedom for the dexterous hand
Parameter description	Handangle.msg int16[6] hand_angle: hand angle array, the range is 0 to 1000, and -1 represents that no operation is performed on this degree of freedom and the current state remains. bool data: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	ros2 topic pub --once /rm_driver/set_hand_angle_cmd rm_ros_interfaces/msg/Handangle "hand_angle: - 0 - 0 - 0 - 0 - 0 block: true"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_hand_angle_result

Setting_the_dexterous_hand_speed

Function description	Setting the dexterous hand speed
Parameter description	Handspeed.msg uint16 hand_speed: hand speed, range: 1-1000.
Command example	ros2 topic pub --once /rm_driver/set_hand_speed_cmd rm_ros_interfaces/msg/Handspeed "hand_speed: 200"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_hand_speed_result

Setting_the_force_threshold_of_the_dexterous_hand

Function description	Setting the force threshold of the dexterous hand
Parameter description	Handforce.msg uint16 hand_force: hand force, range: 1-1000.
Command example	ros2 topic pub --once /rm_driver/set_hand_force_cmd rm_ros_interfaces/msg/Handforce "hand_force: 200"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.

Return example	ros2 topic echo /rm_driver/set_hand_force_result
----------------	---

Setting_the_angle_following_of_the_dexterous_hand

Function description	Setting the angle following of the dexterous hand
Parameter description	Handangle.msg int16[6] hand_angle: hand angle array, the range is 0 to 2000, and -1 represents that no operation is performed on this degree of freedom and the current state remains. bool data: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	ros2 topic pub --once /rm_driver/set_hand_follow_angle_cmd rm_ros_interfaces/msg/Handangle "hand_angle: - 0 - 0 - 0 - 0 - 0 block: true"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_hand_follow_angle_result

Setting_the_posture_following_of_the_dexterous_hand

Function description	Setting the posture following of the dexterous hand
Parameter description	Handangle.msg int16[6] hand_angle: hand posture array, the range is 0 to 1000, and -1 represents that no operation is performed on this degree of freedom and the current state remains. bool data: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	ros2 topic pub --once /rm_driver/set_hand_follow_pos_cmd rm_ros_interfaces/msg/Handangle "hand_angle: - 0 - 0 - 0 - 0 - 0 block: true"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_hand_follow_pos_result

Lifting_mechanism

The RealMan robotic arm can be integrated with the self-developed lifting mechanism.

Speed_open-loop_control_of_the_lifting_mechanism

Function description	Speed open-loop control of the lifting mechanism
Parameter description	Liftspeed.msg int16 speed: speed percentage, -100-100, Speed 0: the lifting mechanism moves upward, Speed = 0: the

	lifting mechanism stops.
Command example	ros2 topic pub /rm_driver/set_lift_speed_cmd rm_ros_interfaces/msg/Liftspeed "speed: 100"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_lift_speed_result

Position_closed-loop_control_of_the_lifting_mechanism

Function description	Position closed-loop control of the lifting mechanism
Parameter description	Liftheight.msg uint16 height: target height, unit: mm, range: 0-2600. uint16 speed: speed percentage, 1-100. bool data: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	ros2 topic pub --once /rm_driver/set_lift_height_cmd rm_ros_interfaces/msg/Liftheight "height: 0 speed: 10 block: true"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_lift_height_result

Get_the_lifting_mechanism_state

Function description	Get the lifting mechanism state
Parameter description	Liftstate.msg int16 height: current height. int16 current: current current. uint16 err_flag: drive error code.
Command example	ros2 topic pub /rm_driver/get_lift_state_cmd std_msgs/msg/Empty "{}"
Return value	Successful return: current state of the lifting mechanism; Failure return: the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/get_lift_state_result

General_expansion_joint

Get_the_state_of_the_expansion_joint

Function description	Get the state of the expansion joint
Parameter description	td_msgs/msg/Empty.
Command example	ros2 topic pub /rm_driver/get_expand_state_cmd std_msgs/msg/Empty "{}"
Return value	Expandstate.msg int16 pos : Angle of the expansion joint, unit: °, accuracy: 0.001°. int16 current : Drive current,Unit: mA, accuracy: 1 mA. uint16 err_flag : Drive error code,Refer to joint error codes for

	details. Current state,mode 0: idle, 1: forward speed motion, 2: forward position motion, 3: backward speed motion, 4: backward position motion.
Return example	ros2 topic echo /rm_driver/get_expand_state_result

Set_the_open-loop_speed_control_of_the_expansion_joint

Function description	Speed open-loop control of the lifting mechanism
Parameter description	Liftspeed.msg int16 speed: speed percentage, -100-100, Speed > 0: the lifting mechanism moves right-hand rule, Speed = 0: the lifting mechanism stops. bool data: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	ros2 topic pub --once /rm_driver/set_expand_speed_cmd std_msgs/msg/Int32 "data: 10"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_expand_speed_result

Set_the_closed-loop_position_control_of_the_expansion_joint

Function description	Set the closed-loop position control of the expansion joint
Parameter description	Expandpos.msg int32 pos: target angle, unit: 0.001°. uint16 speed: speed percentage, 1-100. bool data: whether it is a blocking mode, bool type, true: blocking, false: non-blocking.
Command example	ros2 topic pub --once /rm_driver/set_expand_pos_cmd rm_ros_interfaces/msg/Expandpos "{pos: 100000, speed: 20, block: true}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_expand_pos_result

End_Effector_Ecosystem_Command_Set

Reading of basic and real-time information of end-effector devices supported by the end-effector ecosystem protocol.

Setting_End_Effector_Ecosystem_Protocol_Mode

Function description	Set End-Effector Ecosystem Protocol Mode
Parameter description	std_msgs::msg::Int32 0 - Disable protocol; 9600 - Enable protocol (baud rate 9600); 115200 - Enable protocol (baud rate 115200); 256000 - Enable protocol (baud

	rate 256000); 460800 - Enable protocol (baud rate 460800).
Command example	ros2 topic pub /rm_driver/set_rm_plus_mode_cmd std_msgs/msg/Int32 "data: 0"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_rm_plus_mode_result

Querying_End_Effector_Ecosystem_Protocol_Mode

Function description	Querying End-Effector Ecosystem Protocol Mode
Parameter description	std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/get_rm_plus_mode_cmd std_msgs/msg/Empty "{}"
Return value	0 - Disable protocol; 9600 - Enable protocol (baud rate 9600); 115200 - Enable protocol (baud rate 115200); 256000 - Enable protocol (baud rate 256000); 460800 - Enable protocol (baud rate 460800).
Return example	ros2 topic echo /rm_driver/get_rm_plus_mode_result

Setting Tactile Sensor Mode

Function description	Setting Tactile Sensor Mode
Parameter description	std_msgs::msg::Int32 0 - Disable tactile sensor; 1 - Enable tactile sensor (returns processed data); 2 - Enable tactile sensor (returns raw data).
Command example	ros2 topic pub /rm_driver/set_rm_plus_touch_cmd std_msgs::msg::Int32 "data: 0"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_rm_plus_touch_result

Querying_Tactile_Sensor_Mode

Function description	Querying Tactile Sensor Mode
Parameter description	std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/get_rm_plus_mode_cmd std_msgs/msg/Empty "{}"
Return value	std_msgs::msg::Int32 0 - Disable protocol; 9600 - Enable protocol (baud rate 9600); 115200 - Enable protocol (baud rate 115200); 256000 - Enable protocol (baud rate 256000); 460800 - Enable protocol (baud rate 460800).
Return example	ros2 topic echo

	/rm_driver/get_rm_plus_mode_result
--	------------------------------------

Functions_related_to_the_transmissive_force-position_compensation_Mode

For the RealMan robotic arm with one-axis force and six-axis force versions, the user can not only directly use the teaching device to call the underlying force-position mixing control module but also combine the custom trajectory with the underlying force-position mixing control algorithm in the form of periodic transmission to compensate. If force data calibration has not been completed before the force operations, the zero position can be calibrated using the one-axis force and six-axis force data clear interfaces.

Starting_the_transmissive_force-position_mixing_control_compensation_mode

Function description	starting the transmissive force-position mixing control compensation mode
Parameter description	std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/start_force_position_move_cmd std_msgs/msg/Empty "{}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/start_force_position_move_result

Stopping_the_transmissive_force-position_mixing_control_compensation_mode

Function description	Stopping the transmissive force-position mixing control compensation mode
Parameter description	std_msgs::msg::Empty
Command example	ros2 topic pub /rm_driver/stop_force_position_move_cmd std_msgs/msg/Empty "{}"
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/stop_force_position_move_result

Transmissive_force-position_mixing_control_compensation-joint

Function description	Transmissive force-position mixing control compensation (joint)
Parameter description	Forcepositionmovejoint.msg float32[] joint: target joint radian uint8 sensor: type of sensor used, 0 - one-axis force, 1 - six-axis force uint8 mode, 0 - along the base coordinate system, 1 - along the tool coordinate system. int16 dir: force control direction, 0-5 represent X/Y/Z/Rx/Ry/Rz respectively, where the default direction is Z direction for one-axis force type float32 force: force value, unit: 0.1 N. bool follow: whether high follow, true: high follow, false: low follow. uint8 dof:degree of freedom of the robotic arm
Command example	It needs to be a large number (10 or more) of continuous position points, with more

	than 2ms period continuous release. ros2 topic pub /rm_driver/force_position_move_joint_cmd rm_ros_interfaces/msg/Forcepositionmovejoint " joint: [0, 0, 0, 0, 0, 0] sensor: 0 mode: 0 dir: 0 force: 0.0 follow: false dof: 6
Return value	Success: no return; Failure return: false, and the driver terminal returns an error code.

Transmissive_force-position_mixing_control_compensation-pose

Function description	Transmissive force-position mixing control compensation (pose)
Parameter description	Forcepositionmovepose.msg geometry_msgs/Pose pose: target pose, x, y, z coordinates (float type, unit: m) + quaternion. uint8 sensor: type of sensor used, 0 - one-axis force, 1 - six-axis force. uint8 mode: mode, 0 - along the base coordinate system, 1 - along the tool coordinate system int16 dir: force control direction, 0-5 represent X/Y/Z/Rx/Ry/Rz respectively, where the default direction is Z direction for one-axis force type. float32 force: force value,unit:0.1 N. bool follow: whether high follow, true: high follow, false: low follow.
Command example	It needs to be a large number (10 or more) of continuous position points, with more than 2ms period continuous release. ros2 topic pub /rm_driver/force_position_move_pose_cmd rm_ros_interfaces/msg/Forcepositionmovepose "pose: position: x: 0.0 y: 0.0 z: 0.0 orientation: x: 0.0 y: 0.0 z: 0.0 w: 1.0 sensor: 0 mode: 0 dir: 0 force: 0 follow: false"
Return value	Success: no return; Failure return: false, and the driver terminal returns an error code.

Transmissive_force-position_mixing_control_compensation

Function description	Transmissive_force-position_mixing_control_compensation
Parameter description	Forcepositionmove.msg geometry_msgs/Pose pose #The target pose in the current coordinate system supports quaternion/Euler angle representation for pose. Position accuracy: 0.001mm; Euler angle representation for pose, pose accuracy: 0.001rad; quaternion representation for pose, pose accuracy : 0.000001 float32[] joint #Target joint angle, unit: °, accuracy : 0.001° uint8 flag #0-Issue target angle, 1 - Issue target pose uint8 sensor #0-One-dimensional force; 1-

	six-dimensional force uint8 mode #0- Force control in base coordinate system; force control in tool coordinate system ; uint8[6] control_mode #6 force control directions (Fx, Fy, Fz, Mx, My, Mz) with modes: 0-fixed mode, 1-floating mode, 2-spring mode, 3-motion mode, 4-force tracking mode, 8-force tracking + attitude adaptive mode bool follow #Indicates the motion following effect of the driver, where true indicates high following and false indicates low following. float32[6] desired_force #The desired force/torque maintained by the force control axis will only take effect when the force control mode of the force control axis is set to force tracking mode, with an accuracy of 0.1N. float32[6] limit_vel #The maximum linear velocity and maximum angular velocity limits of the force control axes only take effect when force control is enabled. The maximum linear velocity of the (x, y, z) axes has a precision of 0.001 m/s, and the maximum angular velocity of the (rx, ry, rz) axes has a precision of 0001 °/s uint8 trajectory_mode #In high-following mode, 0 represents full transparent transmission mode, 1 represents curve fitting mode, and 2 represents filtering mode uint16 radio #In curve fitting mode, "radio" is the smoothing coefficient (0-100); in filtering mode, "radio" is the filtering parameter (ranging from 0 to 1000)
Command example	It needs to be a large number (10 or more) of continuous position points, with more than 2ms period continuous release. ros2 topic pub /rm_driver/force_position_move_cmd rm_ros_interfaces/msg/Forcepositionmove "pose: position: x: 0.0 y: 0.0 z: 0.0 orientation: x: 0.0 y: 0.0 z: 0.0 w: 1.0 joint: [] flag: 0 sensor: 0 mode: 0 control_mode: - 0 - 0 - 0 - 0 - 0 - 0 follow: false desired_force: - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 limit_vel: - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 - 0.0 trajectory_mode: 0 radio: 0"
Return value	Success: no return; Failure return: false, and the driver terminal returns an error code.

System_Configuration

Clear_system_errors

Function description	Clear system errors
Parameter description	std_msgs::msg::Empty
Command example	ros2 topic pub --once /rm_driver/clear_system_err_cmd std_msgs/msg/Empty "{}"

Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/clear_system_err_result

Robotic_arm_state_active_reporting

Setting_UDP_robotic_arm_state_active_reporting_configuration

Function description	Set UDP robotic arm state active reporting configuration
Parameter description	Setrealtimepush.msg uint16 cycle: set the broadcast cycle, which is a multiple of 5ms (default 1 i.e. $1 * 5 = 5$ ms, 200 Hz). uint16 port: set the broadcast port number (default 8089). uint16 force_coordinate: set the coordinate system of force data outside the system (only supported by the arm with force sensors). string ip: set the custom reporting target IP address (default 192.168.1.10). bool hand_enable: whether dexterous hand status reporting is enabled, true is enabled, and false is not enabled. aloha_state_enable: whether to enable aloha main arm status reporting, true to enable, false not to enable. arm_current_status_enable: whether to enable the status report of the robot arm, true to enable, false not to enable. expand_state_enable: whether to enable the report of extended joint related data, true is enabled, false is not enabled. joint_speed_enable: whether joint speed reporting is enabled, true is enabled, and false is not enabled. lift_state_enable: whether lifting joint data reporting is enabled, true is enabled, and false is not enabled. plus_base_enable: Basic information of the end-effector device, true is enabled, and false is not enabled. plus_state_enable: Real-time information of the end-effector device, true is enabled, and false is not enabled.
Command example	<pre>ros2 topic pub --once /rm_driver/set_realtime_push_cmd rm_ros_interfaces/msg/Setrealtimepush "cycle: 1 port: 8089 force_coordinate: 0 ip: '192.168.1.10' hand_enable: false aloah_state_enable: false arm_current_status_enable: false expand_state_enable: false joint_speed_enable: false lift_state_enable: false"</pre>
Return value	Successful return: true; failure returns: false, the driver terminal returns an error code.
Return example	ros2 topic echo /rm_driver/set_realtime_push_result

Getting_UDP_robotic_arm_state_active_reporting_configuration

Function description	Get UDP robotic arm state active reporting configuration
Parameter description	Setrealtimepush.msg uint16 cycle: set the broadcast cycle, which is a multiple of 5ms (default 1 i.e. $1 * 5 = 5$ ms, 200 Hz). uint16 port: set the broadcast port number (default 8089). uint16 force_coordinate: set the coordinate system of force data outside the system (only supported by the arm with force sensors). string ip: set the custom reporting target IP address (default 192.168.1.10). bool hand_enable: whether dexterous hand status reporting is enabled, true is enabled, and false is not enabled. aloha_state_enable: whether to enable aloha main arm status reporting, true to enable, false not to enable. arm_current_status_enable: whether to enable the status report of the robot arm, true to enable, false not to enable. expand_state_enable: whether to enable the report of extended joint related data, true is enabled, false is not enabled. joint_speed_enable: whether joint speed reporting is enabled, true is enabled, and false is not enabled. lift_state_enable: whether lifting joint data reporting is enabled, true is enabled, and false is not enabled. plus_base_enable: Basic information of the end-effector device, true is enabled, and false is not enabled. plus_state_enable: Real-time information of the end-effector device, true is enabled, and false is not enabled.
Command example	<pre>ros2 topic pub --once /rm_driver/get_realtime_push_cmd std_msgs/msg/Empty "{}"</pre>
Return value	Successfully set information; Failure return: the driver terminal returns an error code.
Return example	<pre>ros2 topic echo /rm_driver/get_realtime_push_result</pre>

UDP_robotic_arm_state_active_reporting

- Six-axis force

Function description	Six-axis force
Parameter description	Sixforce.msg float32 force_fx: the force along the x-axis direction. float32 force_fy: the force along the y-axis direction. float32 force_fz: the force along the z-axis direction. float32 force_mx: the force when rotating along the x-axis direction. float32 force_my: the force when rotating along the y-axis direction. float32 force_mz: the force when rotating along the z-axis direction.

Subscription command	ros2 topic echo /rm_driver/udp_six_force
----------------------	--

- One-axis force

Function description	One-axis force
Parameter description	Sixforce.msg float32 force_fx: the force along the x-axis direction. float32 force_fy: the force along the y-axis direction. float32 force_fz: the force along the z-axis direction.(only this value is valid) float32 force_mx: the force when rotating along the x-axis direction. float32 force_my: the force when rotating along the y-axis direction. float32 force_mz: the force when rotating along the z-axis direction.
Subscription command	ros2 topic echo /rm_driver/udp_one_force

- Robotic arm error

Function description	Robotic arm error
Parameter description	std_msgs::msg::UInt16 uint16 data: the robotic arm error message.
Subscription command	ros2 topic echo /rm_driver/udp_arm_err

- System error

Function description	System error
Parameter description	std_msgs::msg::UInt16 uint16 data: the system error message.
Subscription command	ros2 topic echo /rm_driver/udp_sys_err

- Joint error

Function description	Joint error
Parameter description	Jointerrorcode.msg uint16[] joint_error: the error message for each joint. UInt8 dof: the arm degree of freedom message.
Subscription command	ros2 topic echo /rm_driver/udp_joint_error_code

- The robot arm radians data

Function description	The robot arm radians data
Parameter description	sensor_msgs::msg::JointState builtin_interfaces/Time stamp int32 sec: time message, unit: second. uint32 nanosec: time message, unit: nanosecond. string frame_id: coordinate system name. string[] name: joint name. float64[] position: joint radian message. float64[] velocity: joint speed message. (not used yet) float64[] effort: joint force message. (not used yet)
Subscription command	ros2 topic echo /joint_states

- Pose information

Function description	Pose information
Parameter description	geometry_msgs::msg::Pose Point position: the robotic arm current coordinate information. float64 x float64 y float64 z Quaternion orientation: the robotic arm current pose information. float64 x 0 float64 y 0 float64 z 0 float64 w 1
Subscription command	ros2 topic echo /rm_driver/udp_arm_position

- Current external force data of the six-axis force sensor system

Function description	Current external force data of the six-axis force sensor system
Parameter description	Sixforce.msg float32 force_fx: the force forced on the current sensor along the x-axis direction. float32 force_fy: the force forced on the current sensor along the y-axis direction. float32 force_fz: the force forced on the current sensor along the z-axis direction. float32 force_mx: the force forced on the current sensor when rotating along the x-axis direction. float32 force_my: the force forced on the current sensor when rotating along the y-axis direction. float32 force_mz: the force forced on the current sensor when rotating along the z-axis direction.
Subscription command	ros2 topic echo /rm_driver/udp_six_zero_force

- Current external force data of the one-axis force sensor system

Function description	Current external force data of the one-axis force sensor system
Parameter description	Sixforce.msg float32 force_fx: the force forced on the current sensor along the x-axis direction. float32 force_fy: the force forced on the current sensor along the y-axis direction. float32 force_fz: the force forced on the current sensor along the z-axis direction. (only this data is valid) float32 force_mx: the force forced on the current sensor when rotating along the x-axis direction. float32 force_my: the force forced on the current sensor when rotating along the y-axis direction. float32 force_mz: the force forced on the current sensor when rotating along the z-axis direction.
Subscription command	ros2 topic echo /rm_driver/udp_one_zero_force

- Reference coordinate system for external force data of the system

Function description	Reference coordinate system for external
----------------------	--

	force data of the system
Parameter description	std_msgs::msg::UInt16 uint16 data: : coordinate system for external force data of the system, where 0 is the sensor coordinate system, 1 is the current work coordinate system, and 2 is the current tool coordinate system This data affects the reference coordinate system for external force data of one-axis and six axis force sensor systems.
Subscription command	ros2 topic echo /rm_driver/udp_arm_coordinate

- The current state of dexterous dexterity

Function description	The current state of dexterous dexterity
Parameter description	rm_ros_interfaces::msg::Handstatus.msg uint16[6] hand_angle: #Finger angle array, range: 0~2000. uint16[6] hand_pos: #Finger position array, range: 0~1000. uint16[6] hand_state: #Finger state, 0 is releasing, 1 is grasping, 2 positions are in place and stop, 3 forces are in place and stop, 5 current protection stops, 6 electric cylinder stalling stops, 7 electric cylinder failure stops. uint16[6] hand_force: #Dexterous hand degree of freedom current, unit mN. uint16 hand_err: #Agile Hand System Error, 1 indicates an error, 0 indicates no error.
Subscription command	ros2 topic echo /rm_driver/udp_hand_status

- current status of the mechanical arm

Function Description	Get the current status of the mechanical arm
parameter description	rm_ros_interfaces:: msg:: Armcurrentstatus.msg uint16 arm_current_status: mechanical arm status 0-RM_IDLE_E // enabled but idle status 1-RM_MOVE_L_E // Move L moving state 2-RM_MOVE_J_E // move J moving state 3-RM_MOVE_C_E // move C moving state 4-RM_MOVE_S_E // move S moving state 5-RM_MOVE_THROUGH_JOINT_E /angle transmission state 6-RM_MOVE_THROUGH_POSE_E // posture transmission state 7-RM_MOVE_THROUGH_FORCE_POSE_E//force control transmission state 8-RM_MOVE_THROUGH_CURRENT_E// Current loop transparent state 9-RM_STOP_E // emergency stop state 10-RM_SLOW_STOP_E // slow stop state 11-RM_PAUSE_E // Pause state 12-RM_CURRENT_DRAG_E // Current loop drag state 13-RM_SENSOR_DRAG_E // Six-axis force drag state 14-

	RM_TECH_DEMONSTRATION_E//Teaching state
query example	ros2 topic echo /rm_driver/udp_arm_current_status

- Current joint current

Function Description	Current Joint Current
parameter description	rm_ros_interfaces:: msg:: Jointcurrent.msg float 32 [] joint_current: current joint current with accuracy of 0.001mA
query example	ros2 topic echo /rm_driver/udp_joint_current

- The current joint enabling state

Function Description	Current Joint Enabling Status
parameter description	rm_ros_interfaces:: msg:: Jointenflag.msg bool [] joint_en_flag: the current joint enabling state, where 1 is the upper enabling state and 0 is the lower enabling state
query example	ros2 topic echo /rm_driver/udp_joint_en_flag

- Euler angle posture of mechanical arm

Function Description	Euler Angle Pose of Mechanical Arm
parameter description	rm_ros_interfaces:: msg:: Jointposeeuler.msg float 32 [3] Euler: Euler angle of current waypoint attitude with accuracy of 0.001rad float32[3] position: current waypoint position with accuracy of 0.000001M
query example	ros2 topic echo /rm_driver/udp_joint_pose_Euler

- Current joint speed

Function Description	Current joint speed
parameter description	rm_ros_interfaces:: msg:: Jointspeed.msg float 32 [] joint_speed: current joint speed with an accuracy of 0.02RPM.
query example	ros2 topic echo /rm_driver/udp_joint_speed

- Current joint temperature

Function Description	Current joint temperature
parameter description	rm_ros_interfaces:: msg:: Jointtemperature.msg float 32 [] joint_temperature: current joint temperature with accuracy of 0.001°C
query example	ros2 topic echo /rm_driver/udp_joint_temperature

- Current joint voltage

Function Description	Current Joint Voltage
parameter description	rm_ros_interfaces::msg::Jointvoltage.msg float 32 [] joint_voltage: current joint voltage with accuracy of 0.001V
query example	ros2 topic echo /rm_driver/udp_joint_voltage

- Lift state

Function Description	Lift state
parameter description	rm_ros_interfaces::msg::Udpliftstate.msg int32 height #Height of the current lifting mechanism Unit: mm, accuracy: 1 mm float32 pos #Current angle Accuracy: 0.001°, unit: ° int16 current #Current drive current Unit: mA, accuracy: 1 mA bool en_flag #Current joint enabling state 1: enable, 0: disable uint16 err_flag #Drive error code.Refer to joint error codes for details
query example	ros2 topic echo /rm_driver/udp_lift_state

- Expand state

Function Description	Expand state
parameter description	rm_ros_interfaces::msg::Udpexpandstate.msg int32 pos #Current angle Accuracy: 0.001°, unit: ° int32 current #Current drive current Unit: mA, accuracy: 1 mA uint16 err_flag #Drive error code.Refer to joint error codes for details. bool en_flag #Current joint enabling state 1: enable, 0: disable uint8 joint_id #Joint ID int16 mode #Current lifting state, 0: idle, 1: forward speed motion, 2: forward position motion, 3: backward speed motion, 4: backward position motion
query example	ros2 topic echo /rm_driver/udp_expand_state

- aloha state

Function Description	aloha state
parameter description	rm_ros_interfaces::msg::AlohaState.msg int16 io1_state #IO1 state (photoelectric detection), 0: key non-triggered, 1: key triggered. int16 io2_state #IO2 state (photoelectric detection), 0: key non-triggered, 1: key triggered.
query example	ros2 topic echo /rm_driver/udp_aloha_state

- Reading Basic Information of End-Effector Device

Function Description	Reading Basic Information of End-Effector Device
----------------------	--

parameter description	rm_ros_interfaces::msg::Rmplusbase.msg string manu:Device manufacturer. int8 type: Device type, including 1 - Two-finger gripper, 2 - Five-finger dexterous hand, 3 - Three-finger gripper string hv:Hardware version string sv:Software version string bv:Bootloader version int32 id:Device ID int8 dof:Degrees of freedom int8 check:Self-check switch int8 bee:Beeper switch bool force:Force control support bool touch:Tactile support int8 touch_num:Number of tactile sensors int8 touch_sw:Tactile switch int8 hand:Hand orientation, including 1 - Left hand, 2 - Right hand int32[12] pos_up:Position upper limit int32[12] pos_low:Position lower limit int32[12] angle_up:Angle upper limit,Unit: 0.01 degrees. int32[12] angle_low:Angle lower limit,Unit: 0.01 degrees. int32[12] speed_up:Speed upper limit int32[12] speed_low:Speed lower limit int32[12] force_up:Force upper limit int32[12] force_low:Force lower limit,Unit: 0.001N.
query example	ros2 topic echo /rm_driver/udp_rm_plus_base

- Reading Real-Time Information of End-Effector Device

Function Description	Reading Real-Time Information of End-Effector Device
parameter description	rm_ros_interfaces::msg::Rmplusstate.msg int32 sys_state:System status. int32[12] dof_state:Current status of each degree of freedom (DoF) int32[12] dof_err>Error information of each DoF int32[12] pos: Current position of each DoF int32[12] speed:Current speed of each DoF int32[12] angle:Current Angle of Each Degree of Freedom angle,Unit: 0.01 degrees. int32[12] current:Current of each DoF,Unit: mA. int32[18] normal_force:Normal force of the tactile three-dimensional force of each DoF int32[18] tangential_force:Tangential force of the tactile three-dimensional force of each DoF int32[18] tangential_force_dir:Direction of the tangential force of the tactile three-dimensional force of each DoF uint32[12] tsa:Tactile self-approach of each DoF uint32[12] tma:Tactile mutual approach of each DoF int32[18] touch_data:Raw data from the tactile sensor int32[12] force:Torque of each DoF,Unit: 0.001N.
query example	ros2 topic echo /rm_driver/udp_rm_plus_state